

A/B Test Meta-Analysis Engine

Portfolio Project Interview Guide

Date: July 08, 2026

Domain: Media and Content Platform (Streaming and News)

Tools: Python, scipy, statsmodels, Power BI

30-Second Elevator Pitch

I built an end-to-end meta-analysis of 100 A/B experiments for a simulated media platform. The project audits experimentation program health using five statistical frameworks: false discovery rate, power analysis, peeking detection, revenue impact modeling, and organizational maturity scoring. I found the program captures only 18% of available value, with \$15M in annual revenue left on the table, and delivered actionable recommendations through a reproducible Python pipeline and Power BI dashboard.

1. Project Objective

The objective was to demonstrate strategic data analyst capabilities beyond running individual t-tests. Instead of analyzing one experiment, this project audits an entire experimentation program to answer: Are we shipping false winners? Are tests underpowered? Is peeking inflating results? How much revenue are we losing? Which experiment types work best? Is the organization improving over time?

This mirrors real work done by senior analysts and experimentation platform teams at companies like Booking.com, Netflix, and Microsoft, who publish research on experimentation program quality.

2. Why This Project Was Built

Problem it solves

- Most data analyst portfolios show descriptive dashboards on clean Kaggle datasets.
- Hiring managers struggle to see if candidates understand experimentation at a strategic level.
- Real A/B test data is proprietary and cannot be shared publicly.

Solution approach

- Simulate 100 realistic experiments with known ground truth (so detection methods can be validated).
- Apply rigorous statistical methods used in industry experimentation audits.
- Translate statistical findings into dollar impact for business stakeholders.

- Deliver reproducible code plus visualization-ready exports for Power BI.

3. What Was Built (End-to-End)

Stage 1: Data Generation

- Python simulation of 100 experiments across 4 teams and 6 feature categories.
- Realistic biases: 42% peeking rate, underpowered sample sizes, conservative shipping.
- Ground truth labels (no_effect, small_positive, medium_positive, negative) for validation.

Stage 2: False Discovery Rate Analysis

- Three FDR estimation methods: ground truth, Benjamini-Hochberg, post-ship reversal.
- Finding: 0% FDR among shipped, but 41% of significant results fail BH correction.

Stage 3: Power Audit

- Median power: 10.9% (target: 80%). 62% of tests severely underpowered.
- 32 of 44 real winners missed. 2.92x more winners detectable at 80% power.

Stage 4: Peeking / P-Hacking Detection

- Caliper test, Fisher's exact test, KS uniformity test, duration shortfall analysis.
- No crude p-hacking, but peeked tests ship 2.1x more often (selection bias).

Stage 5: Revenue Impact

- \$18.4M total opportunity, \$3.3M captured, \$15.0M missed (14.7% of annual revenue).
- \$11.4M additional upside if program reaches 80% power.

Stage 6: Category Patterns + Maturity Score

- Maturity score: 40.2/100 (Early Stage). Notifications has highest detection gap.
- Quarterly learning curve shows flat power improvement, worsening peeking.

Stage 7: Deliverables

- 31 Power BI-ready CSV exports, written findings report, dashboard build guide.

4. Key Findings (Interview-Ready)

Headline metrics

- 100 experiments | 13 shipped (13%) | 17 statistically significant (17%)
- Median statistical power: 10.9% | Peeking rate: 42%
- Revenue capture rate: 18% | Maturity score: 40.2/100

Top 3 insights

- Underpowering is the root cause: teams run tests too short/small to detect real effects.
- Peeking creates selection bias: early-stopped tests ship more often, not because they are better.
- Revenue framing makes the case actionable: \$15M missed is more compelling than 'low power'.

Recommendations delivered

- Implement minimum sample size requirements (lookup table provided).
- Adopt sequential testing for safe peeking.
- Require pre-registration of metrics, duration, and sample size.
- Post-ship holdback validation (5% control traffic for 30 days).

5. Technical Skills Demonstrated

- Python: pandas, numpy, scipy, statsmodels for statistical analysis
- Statistics: two-proportion z-test, Benjamini-Hochberg FDR, power analysis, KS test, chi-squared, Fisher's exact
- Data engineering: reproducible pipeline, modular utils, batch export generation
- Business analytics: revenue modeling, waterfall analysis, maturity scoring
- Visualization: Power BI dashboard design with 5 themed pages
- Communication: executive summary, written report, interview narrative

6. What Interviewers Will Get Out of This

For hiring managers

- Evidence you think beyond SQL queries and basic charts.
- You can connect statistical rigor to business outcomes (dollars, not just p-values).
- You understand how experimentation programs fail at the organizational level.

For technical interviewers

- You can explain WHY each statistical test was chosen and its limitations.
- You built a reproducible pipeline, not a one-off notebook.
- You handled real-world data issues (CSV null parsing, scipy API changes).

For business stakeholders

- Clear ROI case: \$11.4M upside from fixing experimentation infrastructure.
- Prioritized recommendations with expected impact per initiative.
- Maturity score provides a trackable KPI for program improvement.

7. Common Interview Questions & Answers

Q: Why simulated data instead of real?

A: Real A/B results are proprietary. Simulation lets me know ground truth to validate detection methods. Post-ship holdback would be the real-world equivalent.

Q: What is the most actionable finding?

A: The revenue waterfall. It translates process failures into dollars leaders understand. \$15M missed is more actionable than 'median power is 11%'.

Q: What would you do first if hired?

A: Implement minimum sample size requirements using the lookup table. It is low-cost, high-impact, and addresses the root cause of missed winners.

Q: Biggest limitation?

A: Revenue estimates depend on documented assumptions (MAU, ARPU). I would run sensitivity analysis varying those inputs to show robustness.

Q: How is this different from a Kaggle project?

A: It is prescriptive, not descriptive. It answers 'what should we do' not just 'what happened'. It uses multiple data sources, statistical frameworks, and business translation.

8. How to Present in an Interview (5-Minute Structure)

- Minute 1: Context - media platform, 100 experiments, experimentation audit objective.
- Minute 2: Problem - 11% median power, 42% peeking, only 18% revenue captured.
- Minute 3: Approach - 5-stage pipeline, 3 FDR methods, revenue model, maturity score.
- Minute 4: Top finding - \$15M missed, 32 real winners not shipped, 2.92x improvement possible.
- Minute 5: Recommendations + your role - built pipeline, exports, Power BI, written report.

9. Project Repository Structure

data/ - Simulation script and experiments.csv

analysis/ - 5 analysis scripts + run_all.py + utils.py

exports/ - 31 Power BI-ready CSV files

report/ - Findings report, Power BI guide, this interview PDF

README.md - Project overview and setup instructions

10. Next Steps to Strengthen the Portfolio

- Publish Power BI dashboard and add live link to README.
- Add sensitivity analysis section (vary revenue assumptions).
- Record a 3-minute Loom walkthrough of the dashboard.
- Add Bayesian analysis comparison as a bonus section.

This document accompanies the GitHub repository. All analysis is reproducible via: `python analysis/run_all.py`